TRABALHO 2: Desenvolvimento de Sistema de Gestão de Currículos

1. Objetivo

Desenvolver uma aplicação web funcional utilizando o ecossistema **Next.js**, focada na experiência do usuário (UX) e na aplicação de boas práticas de arquitetura de componentes. Nesta primeira etapa, o foco será a construção da interface, gerenciamento de estado local com dados mockados e validação rigorosa de formulários.

2. Requisitos Técnicos e Stack

A aplicação deverá ser construída utilizando obrigatoriamente:

- **Framework:** Next.js (App Router).
- **Estilização:** Tailwind CSS (foco em responsividade).
- **UI Library:** shadcn/ui (componentes de interface).
- **Formulários:** React Hook Form e Yup (validação de esquemas).
- **Máscaras:** React Input Mask Next.
- **Feedback:** Sonner (notificações toast).
- **Microinterações:** Sweet Alert
- **Otimização:** Lazy loadin componentes e dados.
- **Ícones:** React Icon/ Lucide react
- **Imagens:** Next Image Component

3. Arquitetura do Projeto

O projeto deve seguir uma estrutura organizada e modular:

3.1. Componentes Globais

- **Header:** Identificação do sistema e navegação principal.
- **Footer:** Informações de copyright e links secundários.
- **Nav:** Menu de navegação integrado ao Header.

3.2. Rotas e Páginas

1. **Home (`/`):** Landing page apresentando os benefícios do sistema. Aplicar os recursos de UI do shadcn para a landing page.
2. **Lista de Currículos (`/sistema/paginas/curriculos`):** Exibição de cards com resumos (Nome, Cargo, Resumo profissional).
3. **Detalhes do Currículo (`/sistema/paginas/curriculos/[id]`):** Rota dinâmica que exibe todas as informações do candidato e as ações de gestão.
4. **Cadastro (`/sistema/paginas/curriculos/novo`):** Formulário completo para inserção de novos dados. OBS: Fazer upload fake de imagem.

3.3. Proteção de rotas e URL amigável

1. Ajustar as rotas para suprimir o endereço protegendo acessos via exploração
2. As rotas devem seguir o padrão:
    a. **/curriculos/visualizar**
    b. **/curriculos/visualizar/[id]**
    c. **/curriculos/cadastrar**

4. Funcionalidades Esperadas

- **Persistência Mockada:** Os dados devem ser armazenados em constantes ou no `localStorage`. As imagens devem estar dentro da pasta public do projeto.
- **Validação de Dados:** O formulário de cadastro deve validar campos obrigatórios, formato de e-mail e tamanhos mínimos de texto.
- **Máscaras de Entrada:** Aplicar máscaras em campos como CPF, Telefone e Datas.
- **Feedback Visual:** Toda ação (sucesso ao salvar ou erro ao validar) deve disparar um alerta via *Sonner*.

5. Design

O layout deve seguir uma estética **clean**, garantindo:

- **Contraste:** Utilizar paletas de cores análogas, tom sobre tom ou contrastantes que respeitem os níveis de legibilidade.
- **Responsividade:** Adaptação fluida para dispositivos móveis, tablets e desktops.
- **Ícones**: Utilizar ícones que remetem as funcionalidades para uma página interativa.

6. Critérios de Avaliação e Cronograma

Critério	Pontos	Atende	Parcial / Incompleto	Não atende
1. Estrutura e Stack	10	Utiliza corretamente Next.js App Router, Tailwind, shadcn/ui e demais tecnologias solicitadas.	Utiliza a maior parte da stack, mas há tecnologias ausentes ou utilizadas de forma inadequada.	Não utiliza a stack solicitada ou o projeto não apresenta estrutura adequada.
2. Rotas e Navegação	10	Todas as rotas solicitadas funcionam, incluindo rota dinâmica, URLs amigáveis, Header, Nav, Footer e active state.	A maioria das rotas funciona, mas existem problemas ou elementos de navegação incompletos.	Rotas principais não funcionam ou estão ausentes.

3. Landing Page / Home	8	Landing page completa, organizada, responsiva e com boa aplicação dos componentes de UI.	Página existe, mas apresenta pouca elaboração ou problemas de organização/UI.	Página ausente ou sem funcionamento adequado.
4. Listagem de Currículos	8	Cards funcionais com nome, cargo, resumo e acesso aos detalhes.	Listagem funciona, mas apresenta informações ou funcionalidades incompletas.	Não apresenta uma listagem funcional.
5. Página de Detalhes	7	Exibe corretamente todas as informações do currículo através de rota dinâmica.	Exibe parte das informações, mas possui dados ou funcionalidades incompletas.	Página inexistente, quebrada ou sem dados.
6. Formulário de Cadastro	12	Formulário completo, organizado e funcional utilizando React Hook Form.	Formulário funciona parcialmente, mas possui campos ou funcionalidades incompletas.	Formulário ausente ou sem funcionamento.
7. Validação com Yup	8	Valida obrigatórios, e-mail, tamanho mínimo e apresenta mensagens específicas de erro.	Possui validações, mas algumas estão ausentes ou apresentam problemas.	Não utiliza Yup ou não possui validação funcional.
8. Máscaras e Upload	5	CPF, telefone e datas possuem máscaras funcionais e o upload fake está implementado.	Apenas parte das máscaras ou upload está implementada.	Não possui máscaras e/ou upload funcional.
9. Field Arrays — Experiência e Formação	10	Utiliza `useFieldArray`, permitindo adicionar/remover múltiplas experiências e formações, todas validadas pelo Yup.	Implementação parcial: apenas um dos arrays funciona ou há problemas na adição, remoção ou validação.	Não implementa campos dinâmicos.
10. Estados e Feedback	8	Possui Empty State, Skeleton, Sonner para sucesso/erro e feedback específico das validações.	Possui alguns dos estados/feedbacks, mas existem elementos ausentes ou incompletos.	Não implementa os estados e feedbacks solicitados.
11. Busca e Filtro	4	Busca em tempo real funcionando por nome e cargo.	Busca funciona parcialmente ou filtra apenas um dos campos.	Não possui busca/filtro.

12. Responsividade e UI/UX	6	Interface responsiva em mobile, tablet e desktop, com boa hierarquia, contraste e usabilidade.	Interface apresenta responsividade ou problemas de UX em alguns dispositivos.	Interface não é responsiva ou apresenta problemas graves de usabilidade.
13. Microinterações e Estados	2	Botões possuem hover, focus-visible e disabled; microinterações são utilizadas adequadamente.	Alguns estados estão implementados, mas não todos.	Não apresenta os estados solicitados.
14. Componentização e Código	2	Código organizado, componentes reutilizáveis e estrutura clara.	Organização ou componentização apresenta alguns problemas.	Código desorganizado, duplicado ou difícil de manter.
TOTAL	**100**			

7. Entrega

- Repositório do github com o projeto
- Histórico de commits
- Readme
- Deploy vercel ponto extra

8. Desafios Técnicos (Dificuldades Implementadas)

Para atingir a nota máxima na Etapa 1, o projeto deve implementar obrigatoriamente os seguintes desafios:

8.1. Gerenciamento de Formulário Dinâmico (Experiência Profissional)

O formulário de cadastro não deve ser estático. O aluno deve implementar a funcionalidade de **Field Arrays** (Campos Dinâmicos) utilizando o `useFieldArray` do *React Hook Form*:

- O usuário deve ser capaz de adicionar ou remover múltiplos campos de "Experiência Profissional" e "Formação Acadêmica" dinamicamente no mesmo formulário.
- **Validação:** Cada item adicionado dinamicamente deve ser validado pelo esquema do *Yup*.

8.2. Estados de Empty State e Loading Simulado

Para simular o comportamento de uma aplicação real:

- **Empty State:** Caso não existam currículos na constante mockada ou no `localStorage`, a página de listagem deve exibir uma ilustração ou mensagem amigável com um botão de "Criar meu primeiro currículo".
- **Skeleton Screen:** Ao navegar entre as rotas dinâmicas, utilizar o

componente `Skeleton` do *shadcn/ui* para simular o carregamento dos dados antes da renderização final.

8.3. Filtro e Busca em Tempo Real

Na página de lista de currículos (`/curriculos/visualizar`), implementar um campo de busca (Search Bar):

- A lista deve ser filtrada automaticamente por **Nome** ou **Cargo** à medida que o usuário digita (Debounce opcional, mas desejável).

9. Critérios de Refinamento de UI (Tabela de Estilo)

Elemento	Requisito de Dificuldade
Botões	Devem possuir estados de `hover`, `focus-visible` e `disabled` (quando o formulário estiver inválido ou enviando).
Navegação	Implementar um link ativo (`active state`) visual no menu para indicar em qual página o usuário se encontra.
Feedback	O toast do *Sonner* deve conter a descrição do erro específico retornado pela validação do Yup.